



Protecting Removable Storage

INSuRE Fall 2023
Final Presentation

Sponsor: National Security Agency



Project Mentor
Chris DeWeese - cjdewee@nsa.gov

Faculty Support
Brian Ricks - bwr031000@utdallas.edu

Project Team



University of Texas at Dallas Team 2

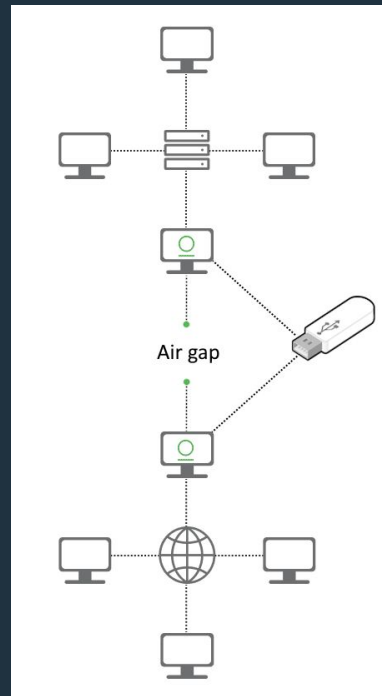
Tri Nguyen	txn180008@utdallas.edu	Computer Science, UTD
Andrei Racila	axr180109@utdallas.edu	Computer Science, UTD
Shreya Soman	svs170430@utdallas.edu	Computer Science, UTD
Shuo-Ting Hsu	sxh200065@utdallas.edu	Computer Science, UTD

Project Motivation



Securing removable storage devices against malware activities and spread.

- Removable storage devices are convenient, capable, and connect to multiple host systems
- Capable of breaching air gap
- Used in virtually all organizations, including governmental and military
- Extensive history as a vector for malware infections



Project Goals



- Understanding the landscape of removable hardware vulnerabilities and defenses
- Implement and evaluate a software solution to mitigate USB-based attacks



Literature Review



USB Attacks Vectors:

1. Abuse of Human Layer:

- Outsider Threats
- Insider Threats

2. Abuse of Application Layer:

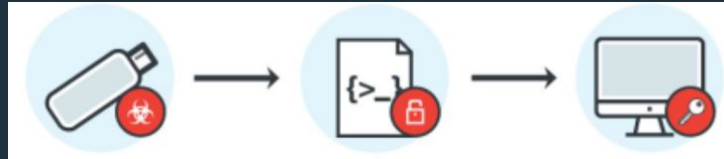
- Code Injection
- Data Exfiltration

3. Abuse of Transport Layer:

- Protocol Masquerading
- Protocol Corruption

4. Abuse of Physical Layer:

- Signal Eavesdropping
- Signal Injection

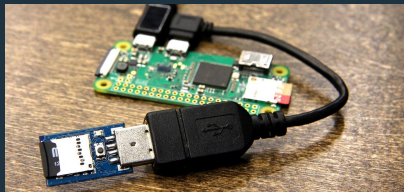


Literature Review



Past USB Attacks

- RubberDucky: emulates the keyboard to inject malicious script
 - USBee, drivebyUSB
 - Psychon: adds keyboard functionality to USB device
- SNOWYDRIVE: create a backdoor, infect other USB devices



Existing Defenses

- White/Blacklisting by device-specific properties
- LBM(Linux eBPF Modules): Filter USB packets in Linux Kernel Space.
- Monitor device behavior in the background
- Fingerprinting



USBGuard & Beamgun

- USB Device authorization via Whitelisting/Blacklisting by vendor, etc.
- Customizable - user can create their own security policy
 - Custom allowable USB list based on properties



Creating an all-rounded system to enhance USB Security!

- Disables suspicious Network Adapters
- Hook keystrokes from USB devices to record keystroke injections
- Steal focus to prevent attacker from making progress



Fingerprinting



- More reliable and generalizable to different forms of removable storage
- Possible fingerprint patterns:
 - User input
 - Utilize feature (product ID, Vendor ID or Serial number)
 - Timing of specific operations
 - Handshake and enumeration process
 - Interfacing with the flash controller



Timing Fingerprint



Read operations on a USB storage device contain enough timing variability to produce a unique fingerprint.

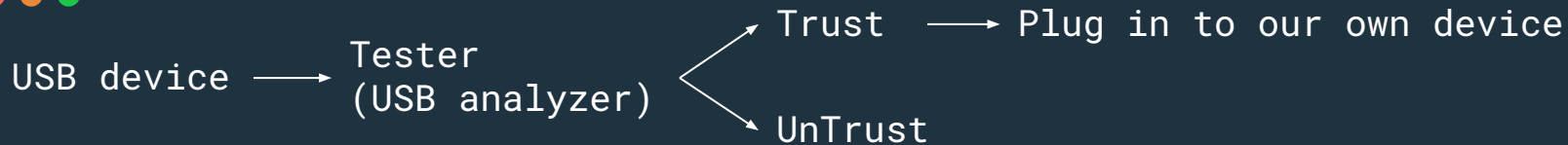
1. Issue a series of read operations to the device
 - a. Communicate with SCSI command
 - b. Utilize `sg_read` utility (sets Direct IO, Disable Page Out and Force Unit Access flags)
2. Precisely record the device's response latency

Timeline



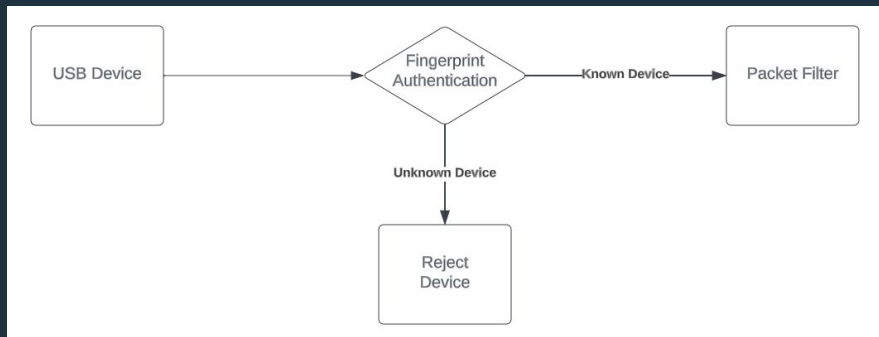
Sep 8, 2023	Proposal
Sep 22, 2023	Phase 1: Literature Review On past USB attacks, USB specifications, and defense frameworks
Oct 6, 2023	Phase 2: Development of Rules Language and Framework Linux eBPF Modules, USBGuard, USB Rubber Ducky
Oct 13, 2023	Midterm Presentation
Nov 17, 2023	Phase 3: Design and Implementation of Solution Timing fingerprint authentication, Packet analysis
Dec 1, 2023	Documentation Summary of findings. Future work.
Dec 8, 2023	Final Presentation

Design



USB analyzer

1. Create Fingerprint for known USB device
2. Fingerprint authentication + Packet filter



Coding - Timing fingerprint

Setup SCSI Read command

```
// Setup Command block
cdb[0] = 0x28;          // SCSI READ opcode
cdb[1] |= (1 << 4);     // Set the Direct IO (DIO) flag
cdb[1] |= (1 << 3);     // Set the Disable Page Out (DPO) flag
cdb[1] |= (1 << 2);     // Set the Force Unit Access (FUA) flag
cdb[2] = (lba >> 24) & 0xFF;
cdb[3] = (lba >> 16) & 0xFF;
cdb[4] = (lba >> 8) & 0xFF;
cdb[5] = lba & 0xFF;
cdb[7] = (read_size >> 8) & 0xFF;
cdb[8] = read_size & 0xFF;

io_hdr.interface_id = 'S';
io_hdr.dxfer_direction = SG_DXFER_FROM_DEV;
io_hdr.cmdp = cdb;
io_hdr.cmd_len = sizeof(cdb);
io_hdr.dxferp = data_buffer;
io_hdr.dxfer_len = read_size;
```

Send the Read operation and record timing with CPU clock cycle

```
asm volatile ( "mrs %0, cntvct_el0" : "=r" (start));
```

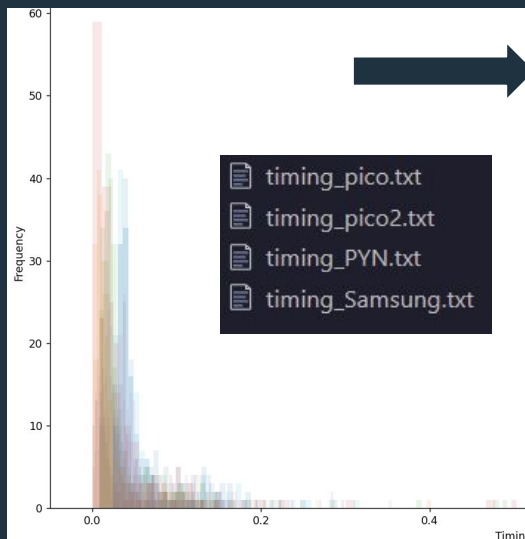
```
ioctl(fd, SG_IO, &io_hdr)
```

```
asm volatile ( "mrs %0, cntvct_el0" : "=r" (end));
```

Coding - Fingerprint Authentication

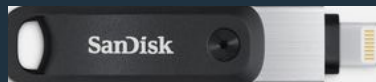


Train a CNN with Timing Data from multiple USB drives



```
# Compile model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy']);
#model.fit here
model.fit(data_X, data_y)
```

“Im a Samsung USB drive”



“No you’re not”

USB Rubber Ducky

- Physical device: Raspberry pi pico
- Firmware: CircuitPython
- Rubber Ducky Capabilities:
 - Automated Keystroke Injection
 - Payload Executions
 - Scripting
- Payload:
 - Remote access control (open reverse shell for remote user to take over the control of victim's computer)
 - Data exfiltration (store the wifi network passphrase from victim's computer)

```
xushuoting@Tims-MacBook-Pro ~ % nc -l 8888
[ (root@kali-raspberry-pi)-[/home/kali]
# ls
ls
Desktop
Documents
Downloads
Music
Pictures
Public
Templates
USB
Videos
```

LBM (Linux eBPF Modules)

- Tool to allow safe interaction between Linux hosts and peripherals (ex.: USB, Bluetooth, etc.)
- Utilizes eBPF alongside a custom language to provide fast filtering of peripheral packets based on user defined rules
- However, it's focus is not on removable storage and is not easily deployable as it's based on a custom Linux kernel

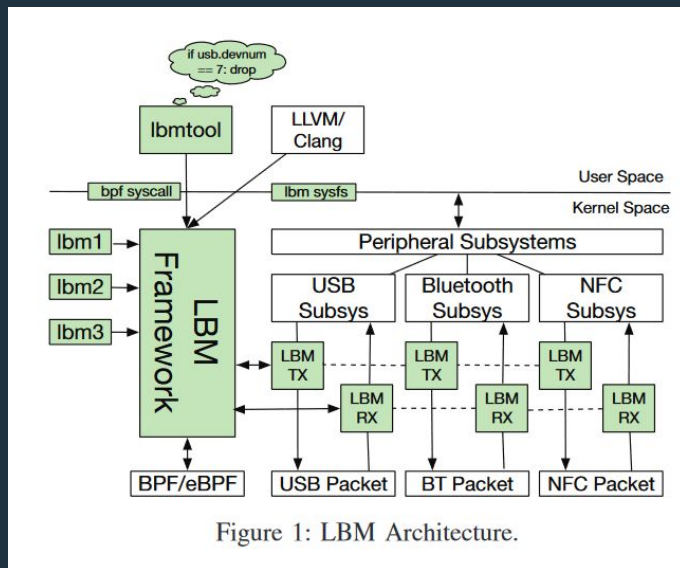


Figure 1: LBM Architecture.

LBM Virtual Machines

- Installed LBM as custom Ubuntu kernel on Linux 16.04 LTS
- Observed interactions between kernel and USB device
- Report information via dmesg (kernel diagnostic message)
- Accessible in /var/log/dmesg file

```
GNU GRUB  version 2.02~beta2-36ubuntu3.1

*Ubuntu, with Linux 4.13.13-lbm
Ubuntu, with Linux 4.13.13-lbm (upstart)
Ubuntu, with Linux 4.13.13-lbm (recovery mode)
Ubuntu, with Linux 4.4.0-210-generic
Ubuntu, with Linux 4.4.0-210-generic (upstart)
Ubuntu, with Linux 4.4.0-210-generic (recovery mode)
Ubuntu, with Linux 4.4.0-31-generic
Ubuntu, with Linux 4.4.0-31-generic (upstart)
Ubuntu, with Linux 4.4.0-31-generic (recovery mode)

Use the ↑ and ↓ keys to select which entry is highlighted.
Press enter to boot the selected OS, 'e' to edit the commands
before booting or 'c' for a command-line. ESC to return previous
menu.

@ubuntu:/var/log$ ls
ves.log  dmesg
```

LBM

- Investigated possibility of lifting out LBM's core functionality out of the kernel source into eBPF programs
 - Faster and wider deployments as wouldn't require re-compiling the kernel
 - Less likely to reintroduce bugs and vulns
 - eBPF programs are ran through verifiers
 - eBPF programs have security guarantees such as guaranteed termination (limited to 4096 instructions), restrictions on modifying kernel memory, and other protections
 - Faster iteration time
 - eBPF programs are loaded on demand
 - Less likely to reintroduce changes that result in kernel panics/crashes

LBM



- eBPF's safety guarantees are also a weakness
 - Ideally want to hook into 2 critical Linux USB handling functions
 - usb_hcd_submit_urb : first kernel function called when USB packets enter the OS from the USB device
 - usb_hcd_giveback_urb : last kernel function called when USB packets move from the OS to the USB device
 - Based on user-defined filtering rules, want to drop packets in these functions i.e change the control flow of these functions
 - eBPF can hook function calls, but only way to change control flow is through a BPF helper function called bpf_override_return
 - "only available if the kernel was compiled with the CONFIG_BPF_KPROBE_OVERRIDE configuration option"
 - "only works on functions tagged with ALLOW_ERROR_INJECTION in the kernel code." (i.e only certain syscalls)

Challenges



- Setting up LBM:
 - LBM was built directly into a 5-year old Ubuntu kernel
 - Compiling the kernel and getting the bootloader (GRUB) to recognize it took a while
 - No code updates since paper release
- Limited knowledge specific to removable storage among team members
- Removable storage protocols are varied and intricate, making the problem space large and more difficult to address fully.

Conclusion

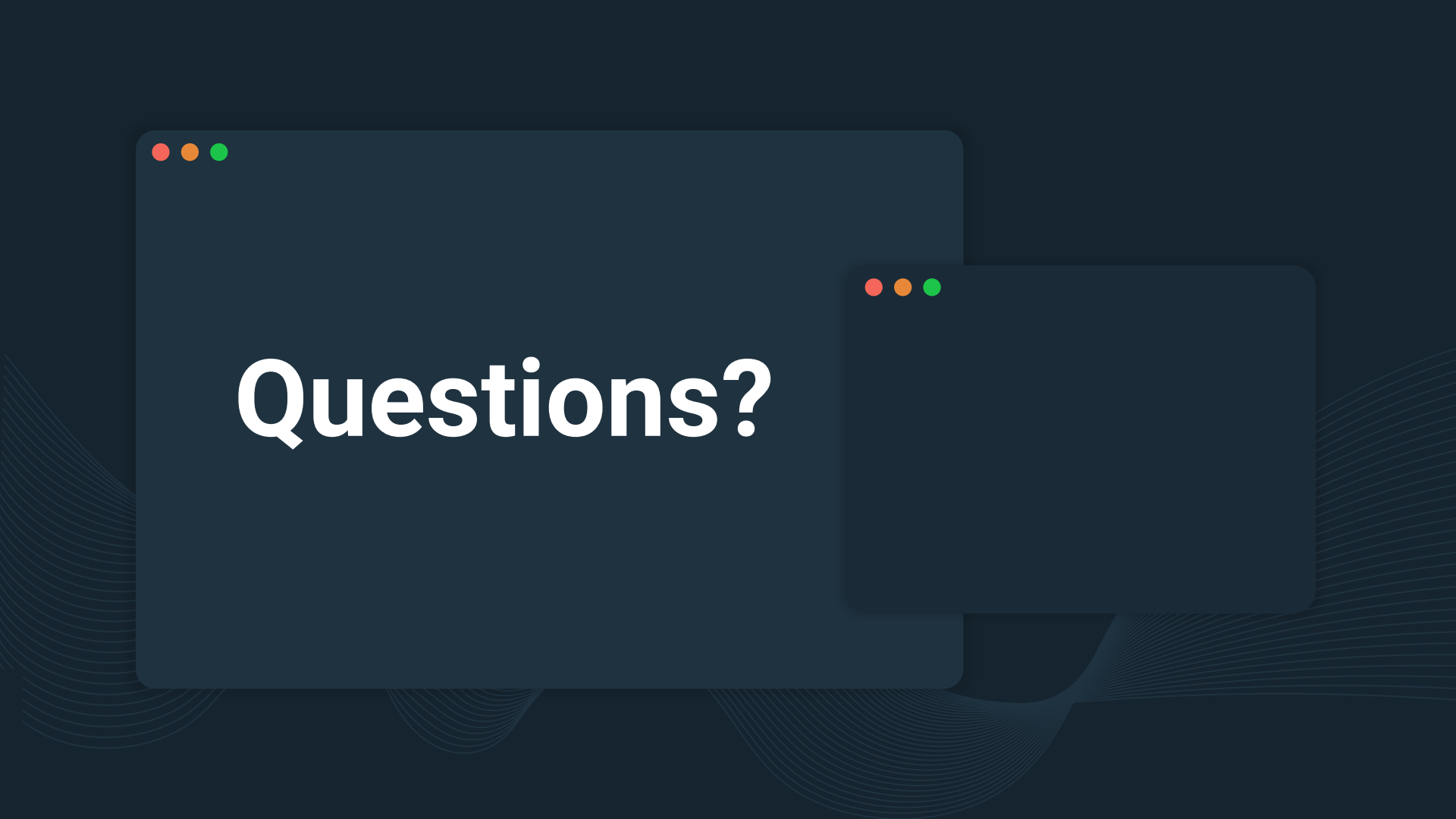
- Summary of Project
- Overview of the variety and severity of badUSB attacks
- Implementation of USB timing fingerprint authentication
- Implementation of badUSB attack for testing
- Understanding of eBPF and LBM packet filtering



Future Work

- Improving defenses:
 - Thorough packet analysis
 - Apply Machine Learning to behavior for fingerprinting
 - Analysis of other layers (physical, etc.)
- Further evaluation and fine tuning:
 - Greater variety of badUSB attacks
 - Creation of standard badUSB test suite
 - Performance optimization
 - Research defenses for more operating systems and removable storage protocols





Questions?